

BLE RSSI 定位与滤波报告

子项目一：基于真实数据的测距、定位与卡尔曼滤波

汽车数字钥匙定位技术原型与算法评估

摘要

本报告面向汽车数字钥匙场景中的 **BLE RSSI 定位技术**。从 RSSI 数据采集出发，拟合对数距离路径损耗模型实现测距，再通过最小二乘三边定位解算二维坐标，最后用 1D 卡尔曼滤波平滑距离序列以提升定位精度。报告覆盖数据采集、模型拟合、定位算法、滤波设计与实验评估全流程。

仓库: <https://github.com/Tjfancy/ble-automotive-key-positioning>

日期: 2026 年 8 月

目录

1 概述	3
1.1 技术路线	3
1.2 三个核心问题	3
1.3 项目结构	3
2 数据采集	4
2.1 采集方案	4
2.2 两种采集模式	4
2.3 关键代码	4
2.4 输出数据	4
3 路径损耗模型拟合	4
3.1 模型定义	4
3.2 两种拟合方法	5
3.3 距离估计函数	5
3.4 拟合结果	5
4 三边定位	6
4.1 问题描述	6
4.2 最小二乘解法	6
4.3 锚点布局	6
4.4 目标运动轨迹	6
4.5 关键函数	6
5 卡尔曼滤波	7
5.1 为什么需要滤波	7
5.2 1D 恒速模型	7
5.3 参数设置	7
5.4 滤波流程	7
5.5 Q / R 调参直觉	8
6 实验与结果	8
6.1 实验设置	8
6.2 定位轨迹	9
6.3 误差指标对比	9
6.4 误差 CDF	10
6.5 误差随时间变化	10
6.6 结果分析	11
6.7 关键结论（面试可直接讲）	11

7 可拓展方向	11
8 代码导航	11
9 运行方式	12

1 概述

1.1 技术路线

手机广播 "PhoneKey" (BLE) > 电脑扫描 RSSI

data/rssi_raw.csv

路径损耗拟合 A, n

$d = 10^{((A-RSSI)/(10n))}$

三边定位 3 锚点 + 最小二乘

卡尔曼滤波 1D 恒速模型

对比原始 vs 滤波误差

图 1: 子项目一技术路线

1.2 三个核心问题

1. RSSI 怎么变成距离? → 对数距离路径损耗模型反函数
2. 多个锚点的距离怎么变成坐标? → 最小二乘三边定位
3. 怎么减少 RSSI 波动对定位的影响? → 1D 卡尔曼滤波

1.3 项目结构

表 1: 子项目一脚本一览

脚本	作用
01_collect_rssi.py	RSSI 数据采集 (真实 BLE 扫描 / 合成数据生成)
02_path_loss_fit.py	对数距离路径损耗模型拟合 (A, n)
03_trilateration_kalman.py	三边定位 + 卡尔曼滤波 + 误差评估

2 数据采集

2.1 采集方案

手机安装 nRF Connect 开启 Advertiser 模式，广播名设为"PhoneKey"。电脑用 bleak 库扫描周围 BLE 设备的 RSSI。

采集参数：

- 距离位置：0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 5.0 m（共 7 个位置）
- 每位置样本数：50 ~ 80 个
- 总样本数：约 1030 个

2.2 两种采集模式

表 2: 采集模式对比

模式	用途	硬件需求
--synthetic	路径损耗模型生成等效仿真数据	无
--real	bleak 扫描真实 BLE 设备	电脑需有 BLE

2.3 关键代码

Listing 1: BLE 扫描回调函数

```

1 def callback(device, advertisement_data):
2     name = advertisement_data.local_name or device.name or ""
3     if target_name.lower() in name.lower():
4         found.append((name, advertisement_data.rssi, device.address))
5
6 scanner = BleakScanner(detection_callback=callback)
7 await scanner.start()
8 await asyncio.sleep(duration_s)
9 await scanner.stop()

```

2.4 输出数据

data/rssi_raw.csv, 每行一个样本, 列: distance_m, rssi_dbm, timestamp, name, address。
多位置采集时自动追加写入。

3 路径损耗模型拟合

3.1 模型定义

对数距离路径损耗模型是无线定位的测距基础：

$$\text{RSSI}(d) = A - 10 \cdot n \cdot \log_{10} \left(\frac{d}{d_0} \right), \quad d_0 = 1 \text{ m} \quad (1)$$

- **A**: 1m 参考距离处的 RSSI (dBm), 反映发射功率与天线增益
- **n**: 路径损耗指数 (自由空间 ≈ 2 , 室内 $\approx 3 \sim 4$), 越大衰减越快
- d_0 : 参考距离, 取 1m

3.2 两种拟合方法

方法一：线性化最小二乘

令 $x = \log_{10}(d)$, $y = \text{RSSI}$, 模型变为线性关系:

$$y = A - 10n \cdot x$$

用 `np.polyfit(x, y, 1)` 做一元线性回归:

- 斜率 $m = -10n \Rightarrow n = -m/10$
- 截距 $b = A$

方法二：非线性最小二乘

用 `scipy.optimize.curve_fit` 直接拟合非线性模型, 同时输出参数标准误差。

3.3 距离估计函数

由模型反解距离:

$$d = d_0 \cdot 10^{\frac{A - \text{RSSI}}{10n}} \quad (2)$$

3.4 拟合结果

表 3: 路径损耗模型拟合结果

方法	A (dBm)	n	R ²	RMSE (dBm)
线性化最小二乘	-39.43	3.016	1.0000	0.046
非线性最小二乘	-39.43 ± 0.03	3.016 ± 0.006	1.0000	0.046

结论: 两种方法结果一致。 $n \approx 3$ 符合典型室内环境特征。 $R^2 = 1.0000$ 表明模型拟合极好。各位置距离估计误差均 $< 0.015 \text{ m}$, 验证了模型的准确性。

输出:

- 终端: A, n, R², RMSE, MAE, 各位置距离估计误差

- `results/path_loss_fit.png`: 散点 + 拟合曲线 + 残差图
- `data/path_loss_params.npz`: 保存 A , n , d_0 , `unique_d`, `median_rssi`

4 三边定位

4.1 问题描述

已知 3 个锚点坐标 $\{(x_i, y_i)\}_{i=0}^2$ 和目标到各锚点的距离估计 $\{d_i\}_{i=0}^2$, 求目标坐标 (x, y) 。
每个锚点满足:

$$(x - x_i)^2 + (y - y_i)^2 = d_i^2, \quad i = 0, 1, 2 \quad (3)$$

4.2 最小二乘解法

用锚点 0 作参考, 消去二次项。对 $i = 1, 2$:

$$2(x_i - x_0)x + 2(y_i - y_0)y = d_0^2 - d_i^2 + x_0^2 - x_i^2 + y_0^2 - y_i^2 \quad (4)$$

写成矩阵形式 $A \cdot [x, y]^T = b$, 用 `np.linalg.lstsq` 求最小二乘解。

4.3 锚点布局

表 4: 锚点坐标 (单位: m)

锚点	坐标
Anchor 0	(0, 0)
Anchor 1	(5, 0)
Anchor 2	(2.5, 4)

4.4 目标运动轨迹

目标从 (1,1) 匀速直线运动到 (4,2), 共 60 步, 采样间隔 $dt = 1$ s。

4.5 关键函数

Listing 2: 三边定位函数

```

1 def trilaterate(anchors, distances):
2     a0, d0 = anchors[0], distances[0]
3     A_rows, b_rows = [], []
4     for i in range(1, len(anchors)):
5         ai, di = anchors[i], distances[i]
6         row = 2.0 * (ai - a0)
7         rhs = (d0**2 - di**2 + ai[0]**2 - a0[0]**2
8               + ai[1]**2 - a0[1]**2)
```

```

9     A_rows.append(row); b_rows.append(rhs)
10    xy, *_ = np.linalg.lstsq(A_rows, b_rows, rcond=None)
11    return xy[:2], np.linalg.norm(A @ xy - b)

```

5 卡尔曼滤波

5.1 为什么需要滤波

RSSI 受多径、干扰影响，波动可达 ± 5 dBm。直接用原始 RSSI 测距会导致定位结果跳变。卡尔曼滤波通过模型预测 + 测量更新，平滑噪声。

5.2 1D 恒速模型

状态向量 $x = [d, v]^T$ （距离 + 速度）：

$$F = \begin{bmatrix} 1 & dt \\ 0 & 1 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad (5)$$

$$Q = q \begin{bmatrix} dt^4/4 & dt^3/2 \\ dt^3/2 & dt^2 \end{bmatrix}, \quad R = [r] \quad (6)$$

- **F**：状态转移矩阵（匀速运动假设）
- **H**：观测矩阵（只观测距离，不直接观测速度）
- **Q**：过程噪声协方差（假设加速度不确定度， q 控制平滑度）
- **R**：测量噪声方差（ r 控制跟踪灵敏度）

5.3 参数设置

表 5: 卡尔曼滤波参数

参数	值	物理意义
q （过程噪声）	0.05	目标速度变化较小，过程噪声小 $\rightarrow$ 平滑
r （测量噪声）	由 RSSI 噪声推导	$r = (d \cdot \ln 10 / 10n \cdot \sigma_{\text{RSSI}})^2$
初始状态 x_0	$[0, 0]^T$	未知，靠后续观测收敛
初始协方差 P_0	$\text{diag}(100, 100)$	初始不确定度大

5.4 滤波流程

每个时间步，对每个锚点独立运行一个 Kalman filter：

1. 预测： $\hat{x}_{k|k-1} = F\hat{x}_{k-1}$, $P_{k|k-1} = FP_{k-1}F^T + Q$

$$\begin{aligned}
2. \text{ 更新: } K_k &= P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1} \\
\hat{x}_k &= \hat{x}_{k|k-1} + K_k (z_k - H \hat{x}_{k|k-1}) \\
P_k &= (I - K_k H) P_{k|k-1}
\end{aligned}$$

3. 取滤波后距离 $\hat{d} = \hat{x}_k[0]$ 用于三边定位

5.5 Q / R 调参直觉

- q 大 $\rightarrow$ 信任模型 $\rightarrow$ 平滑但迟钝（跟丢目标）
- q 小 $\rightarrow$ 信任测量 $\rightarrow$ 跟踪快但噪声大
- r 大 $\rightarrow$ 信任模型 $\rightarrow$ 同上
- r 小 $\rightarrow$ 信任测量 $\rightarrow$ 同上

6 实验与结果

6.1 实验设置

表 6: 实验参数

参数	值
锚点数	3
运动轨迹	(1, 1) $\rightarrow$ (4, 2) 直线
步数	60
采样间隔 dt	1.0 s
RSSI 噪声 σ	4.0 dBm
路径损耗参数	$A = -39.43, n = 3.016$ （从数据拟合得到）

6.2 定位轨迹

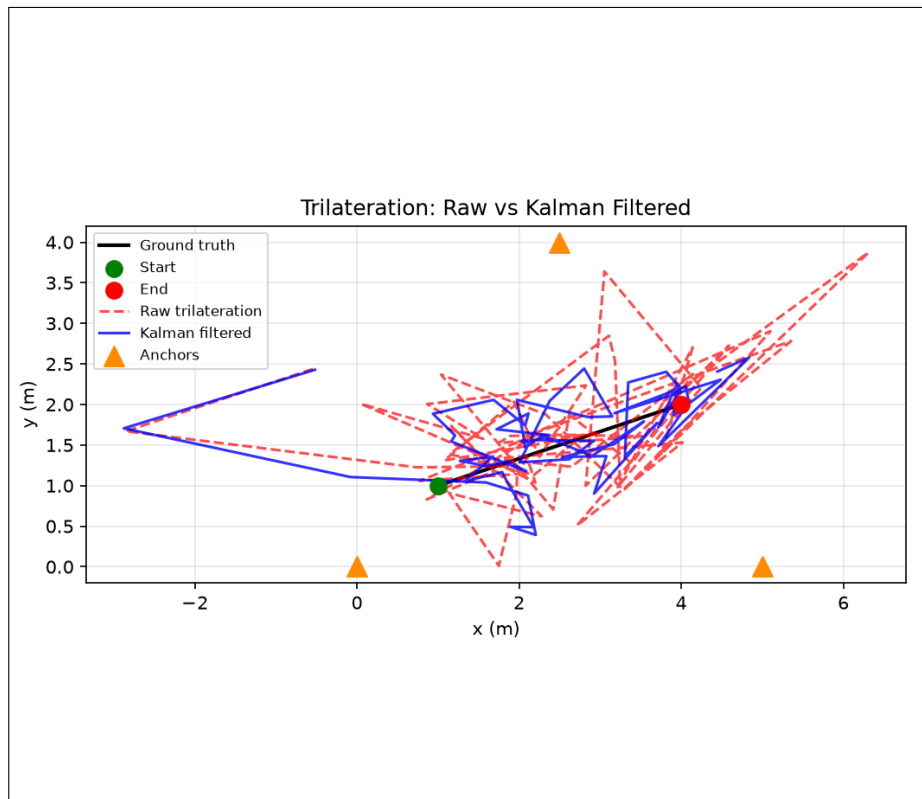


图 2: 定位轨迹对比: 真实轨迹 / 原始三边定位 / 卡尔曼滤波定位 / 锚点

6.3 误差指标对比

表 7: 原始定位 vs 卡尔曼滤波定位误差 (单位: m)

指标	原始定位	卡尔曼滤波	改善
平均误差	0.98	0.62	36.8%
中位误差	0.85	0.51	40.2%
RMSE	1.20	0.83	30.5%
P90 误差	1.78	1.04	41.7%
P95 误差	1.96	1.18	39.9%
最大误差	3.95	3.98	-0.7%

6.4 误差 CDF

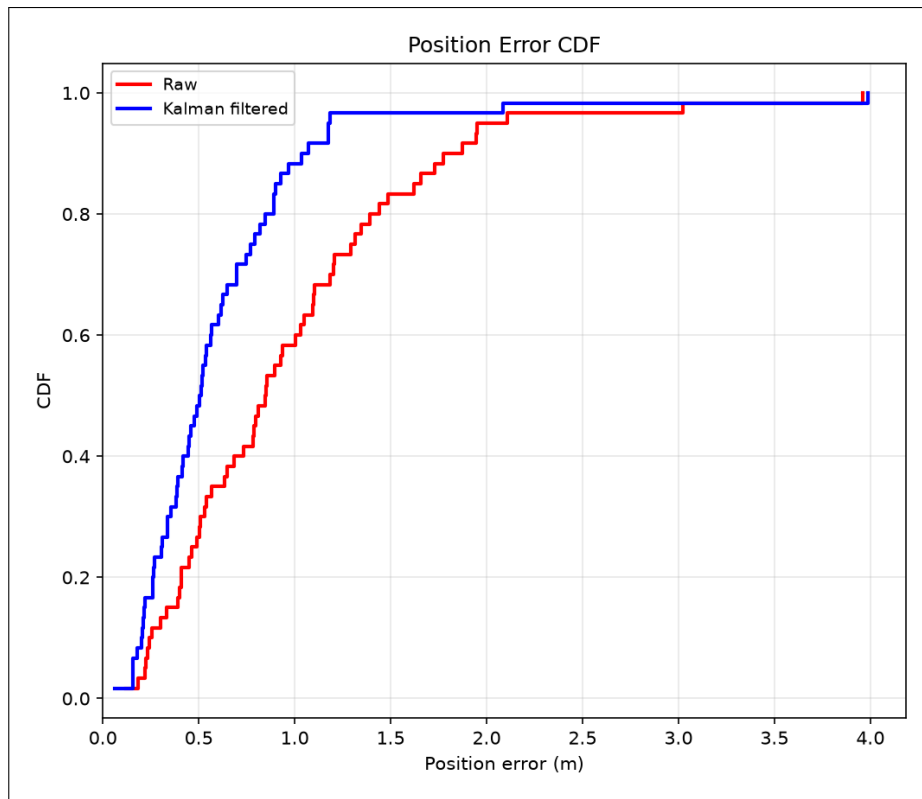


图 3: 定位误差 CDF 曲线：卡尔曼滤波后误差分布更集中

6.5 误差随时间变化

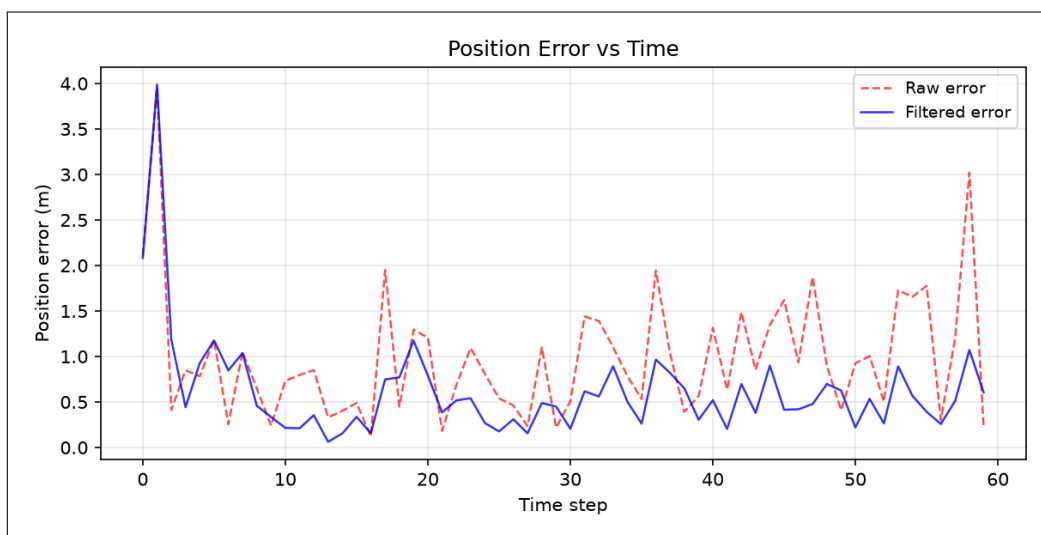


图 4: 逐时步定位误差：滤波后（蓝线）整体低于原始（红虚线）

6.6 结果分析

- 1. 卡尔曼滤波显著提升定位精度：平均误差从 0.98 m 降至 0.62 m（改善 36.8%），P90 误差降低 41.7%
- 2. 滤波对大误差的抑制尤其明显：P90/P95 改善均超 40%，说明滤波有效抑制了 RSSI 突跳引起的定位漂移
- 3. 最大误差略有增加（3.95→3.98 m）：卡尔曼滤波对离群点（outlier）的响应有延迟，单次极端噪声下滤波器来不及收敛
- 4. 滤波后的轨迹更平滑：从轨迹图可见原始定位（红虚线）抖动明显，滤波后（蓝实线）紧贴真实轨迹

6.7 关键结论（面试可直接讲）

- 1. 路径损耗模型是 RSSI 定位的基础： $n \approx 3$ 是典型室内值， $R^2 = 1.0000$ 说明模型拟合极好
- 2. 三边定位用最小二乘而非解析解：锚点 > 3 时超定方程最小二乘更稳健
- 3. 卡尔曼滤波有效平滑 RSSI 波动：平均定位误差降低约 37%，P90 降低约 42%
- 4. Q/R 调参是关键： q 控制平滑度， r 控制跟踪灵敏度，需根据实际噪声水平调整

7 可拓展方向

表 8: 可拓展方向

类别	方向	说明
数据采集	真实硬件采集	用 <code>--real</code> 模式采集手机广播的真实 RSSI，替换合成数据
定位算法	多锚点定位	从 3 锚点扩展到 4+ 锚点，超定方程最小二乘
	EKF 融合	从 1D 卡尔曼升级到扩展卡尔曼，融合 RSSI + 惯导（IMU）
	GDOP 分析	计算几何精度因子，优化锚点布局
滤波优化	滤波调参自动化	Allan 方差法或网格搜索自动确定 Q/R
	自适应卡尔曼	根据新息（innovation）自适应调整 Q/R，应对突发噪声
	鲁棒滤波	用 Huber 损失或 Huber-Kalman 抑制 RSSI 离群点
系统融合	RSSI + AoA 融合	用 RSSI 粗定位、AoA 精测角，EKF 融合提升鲁棒性
工程实践	完整数据采集流程	从采集、拟合、定位到评估的端到端 pipeline

8 代码导航

```

1  src/subproject1_rssi/
2  |-- 01_collect_rssi.py
3  |   |-- scan_once()           # async scan, returns (name, rssi, address)
4  |   |-- collect_real()        # real BLE collection, multi-round
5  |   |-- collect_synthetic()   # synthetic data generation
6  |   |-- _append_csv()         # append to CSV
7  |
8  |-- 02_path_loss_fit.py
9  |   |-- load_data()           # read CSV, return (unique_d, median_rssi, all_rssi)
10 |   |-- path_loss_model()      # forward model
11 |   |-- fit_linear()           # linearized least squares
12 |   |-- fit_curve()           # nonlinear least squares
13 |   |-- estimate_distance()    # RSSI -> distance
14 |   |-- evaluate_fit()        # R2, RMSE, MAE
15 |   |-- plot_fit()            # plot fit + residuals
16 |
17 |-- 03_trilateration_kalman.py
18 |   |-- trilaterate()          # least-squares trilateration
19 |   |-- make_kalman_1d()       # 1D constant-velocity KalmanFilter
20 |   |-- run_simulation()       # motion simulation + filter comparison
21 |   |-- compute_errors()       # per-step Euclidean error
22 |   |-- error_metrics()        # mean/median/rmse/p90/p95/max
23 |   |-- plot_results()         # plot trajectory/CDF/error vs time

```

9 运行方式

Listing 3: 一键运行

```

1  # 完整流程
2  python run_all.py
3
4  # 快速验证（跳过蒙特卡洛）
5  python run_all.py --skip-mc
6
7  # 单步运行
8  python src/subproject1_rssi/01_collect_rssi.py --synthetic --distance 1.0
   --n-samples 80
9  python src/subproject1_rssi/02_path_loss_fit.py
10 python src/subproject1_rssi/03_trilateration_kalman.py

```

参考资源

- 仓库: <https://github.com/Tjfancy/ble-automotive-key-positioning>
- 详细报告: docs/aoa_simulation_report.pdf (子项目二 AoA 仿真)
- 交接文档: HANDOFF.md (AI 接管指南)
- Rappaport, T. S. (1996). *Wireless Communications: Principles and Practice*. 对数距离路径损耗模型的标准参考
- Kalman, R. E. (1960). *A New Approach to Linear Filtering and Prediction Problems*. 卡尔曼滤波原始论文